BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

It is impossible to create a mathematically perfect Table-Driven Agent for a complex environment such as Chess because the number of possible percept sequences becomes extremely large. A table-driven agent would need to store an action for every possible sequence of percepts that the agent could receive throughout its lifetime. In Chess, there are an enormous number of possible board positions, moves, and move sequences, so the table would become unimaginably large. This is called combinatorial explosion. As the agent's lifetime increases, the length of its percept sequence also increases, causing the number of possible histories to grow exponentially. Therefore,

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The SimpleReflexAgent directly follows the Condition-Action Rule architecture because it checks the current percept and immediately selects an action without using any memory of previous situations. For example, the statement if percept['food_here']: represents the rule IF food is here THEN perform the corresponding action. Similarly, elif percept['wall_ahead']: represents IF there is a wall ahead THEN turn left, while the else statement represents IF none of the previous conditions are true THEN move forward. These IF-THEN decisions directly map the agent's current percept to an action. Therefore, the code demonstrates a simple reflex agent because the agent makes its

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The SimpleReflexAgent can get stuck in an infinite loop because the environment is partially observable, meaning the agent cannot see the complete state of the environment. It only receives limited information such as whether there is a wall ahead or food nearby. In addition, the Simple Reflex Agent does not maintain any percept history or internal memory. Therefore, when it reaches the same situation again, it cannot recognize that it has already experienced that situation. It applies exactly the same condition-action rule again and produces the same action. For example, the agent may turn left when it detects a wall, move forward, encounter another wall, turn left again, and eventually return to a previously visited position. Since it has no memory to recognize the repeated situation, it continues performing the same sequence of

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The ModelBasedAgent improves the Simple Reflex Agent by maintaining an internal state or memory of the environment. The Sensor Model describes how the actual environment produces the percept received by the agent. In this implementation, get_percept() provides limited information such as whether there is a wall ahead or food at the current location. The Transition Model describes how the environment changes when an action is performed. For example, when the agent moves forward or turns, its position or direction changes, and the environment may also change when food is collected or opponents move. The Model-Based Agent records information such as visited cells, its current position, facing direction, and previous action, allowing it to maintain an internal representation of the world. It can then use this memory when selecting its next action, such as avoiding a previously visited path or choosing an alternative direction when it detects that it is entering a loop. Therefore, unlike the

Finalize and Lock Lab 02 Documentation

FACULTY OF COMPUTING